MatchPulse — Architecture Characteristics

This document derives the architecture characteristics of MatchPulse from the kata, selects a minimal set, and states how each one is quantified. It deliberately does **not** claim that the system maximises every characteristic. The selection follows the Software Architectures course's characteristic vocabulary and the task requirement to relate each characteristic to specific points of the kata.

A dated synthesis of the kata, characteristics, architecture style and evidence is in `reports/2026-11-08.md`.

1. Status vocabulary

Every claim carries one of these labels. A characteristic with no evidence is labelled, never implied.

Label	Meaning
Specified	A target is defined and agreed. No observation yet.
Measured	An observation exists, with a named environment and scope.
Unverified	Asserted somewhere in the repository with no supporting evidence.
Not implemented	The requirement is documented; the code does not enforce it.

2. Course vocabulary and selection

The course presents a consolidated 12-item table: overall cost, partitioning type, simplicity, modularity, maintainability, testability, deployability, evolvability, responsiveness, scalability, elasticity and fault tolerance. The same course also discusses **Reliability** and **Performance** as operational characteristics, and explicitly says the list is non-exhaustive and questionable.

Two naming decisions follow from that:

- **Reliability** is used for the correctness of concurrent operations. It is a course operational characteristic, even though the consolidated 12-item table uses fault tolerance and responsiveness. Fault tolerance is about operating through hardware or software faults; it is not the right name for a capacity or balance invariant under simultaneous requests.
- **Security** is used as an **implicit cross-cutting characteristic**. It is not in the 12-item table, but the course's implicit-characteristics slide uses security as the example: stakeholders rarely state it, and the architect must make it explicit. The kata's staff-only management, points ledger and club privacy requirement make it explicit here.

Course characteristic	Selected?	Kata points	Reason
Overall cost	No	K1–K15	Cost is a constraint of a solo project, not a quantified architecture target.
Partitioning type	No	K1–K14	Domain-oriented database modules with a layered delivery shell is a design choice, not a separately measured quality.
Simplicity	No	K1–K15	The modular monolith is chosen for simplicity, but there is no separate simplicity metric.
Modularity	No	K2, K4, K6–K12	Module boundaries are a means to reliability and testability; no requirement forces independent module evolution.
Maintainability	No	K1–K15	The idempotency change demonstrates it, but it is not a primary success driver for the first increment.
Testability	**Yes**	K2, K4, K6–K8, K10–K12	The central rules are concurrency rules and cannot be verified by inspection.
Deployability	**Yes**	K1–K14; solo-developer context	A one-person project across two deadlines must deliver increments without infrastructure work.
Evolvability	No	K1–K15	The change scenario exercises it, but no separate evolution target is required for the first increment.
Responsiveness	No	K5, K6	The only time target is the burst-response p95 under elasticity, not a general page or API target.
Scalability	No	K5, K6	No requirement asks for growth beyond match-time peaks; elasticity covers the burst.
Elasticity	**Yes**	K5, K6 and the traffic context	Match-time traffic arrives as a step change, not a gradual ramp.
Fault tolerance	No	K1–K12	There is no hardware-fault or availability target; reliability covers concurrent correctness.
Reliability	**Yes**	K2, K4, K6–K8, K10–K12	Capacity, balances, positions and settlement must stay correct under simultaneous requests.
Security (implicit)	**Yes**	K1, K13, K14	Staff-only management, the points ledger and club privacy require an authorization boundary.

Five characteristics are selected: **Reliability, Elasticity, Deployability, Testability and Security**. The selection criterion is that failing the characteristic would break a documented business rule or make progress unverifiable.

3. Selected characteristics

3.1 Reliability — integrity under concurrency

Course definition. Reliability is the degree to which a system functions under specified conditions for a specified period of time (`SA_Lecture5`). The conditions and the period matter as much as the quality itself.

Kata points. K2 (capacity), K4 (registration while capacity remains), K6–K8 (entry, exit and proportional settlement), K10 (cooldown without blocking exits), K11–K12 (result finalisation and one-shot settlement). The additional context states that participation must never exceed capacity, including simultaneous requests, and that balances must remain consistent under concurrent requests.

Why it matters. Every rule that moves points is a concurrency rule. A failure is silent: it corrupts the ledger or overbooks a tournament instead of raising a visible error. The system cannot be trusted if the rules hold only when requests arrive one at a time.

SMART target.

Criterion	Target
Specific	Under a burst of 100 concurrent registrations against a tournament of capacity 32, exactly 32 are accepted, 68 are refused for a business reason, 0 requests fail at the transport level, and committed state matches the accepted count. Concurrent entries cannot make a balance negative. A resolved market pays each winning share once. A retried entry or exit with the same request key does not duplicate a trade.
Measurable	The invariant suite asserts response counts, committed rows, the stored counter, balances, positions, trade rows and request-key replay.
Assignable	Project owner (solo developer).
Realistic	The scenario uses one local PostgreSQL instance and 100 authenticated clients, a scale the project can run in CI.
Time-related	Verified in CI by 2026-11-08; re-checked at every release and before the project submission.

Measurement method. `tests/invariants/tournament-capacity.test.ts` issues 100 concurrent `register_for_tournament` calls against capacity 32 and asserts the committed state. `market-concurrency.test.ts` and `market-pricing.test.ts` assert balance and position consistency. `idempotency.test.ts` repeats entries and exits with the same request key. `market-lifecycle.test.ts` asserts one-shot resolution. The enforcing mechanisms are the tournament-row lock, the profile lock and conditional debit, the position constraints, the terminal market state and the `trade_requests` unique key.

Evidence. CI run 42435627680 passed 28 tests with 0 failures, including the capacity, concurrency, authorization and idempotency suites. The profile lock (M-INV4), terminal resolution (M-INV6) and the entry-cooldown gate (M-INV7) were previously observed failing under fault injection. The capacity guard and uniqueness constraint were also observed failing in the Checkpoint A evidence.

Status. Measured for the recorded mechanisms, with one explicit gap. The remaining market invariants (M-INV1, M-INV2, M-INV3, M-INV5, M-INV8 and the M-INV7 ladder details) have passing tests but no observed-red injection; they remain **Enforced, unverified** in the invariant registry (FND-002). No payout path is called verified from a green run alone.

3.2 Elasticity

Course definition. Elasticity is the ability to handle bursts of users. The course's measurement includes two parts: the system replies in X ms under a peak of Y users, and the percentage of resources effectively utilised after the peak (`SA_Lecture2`).

Kata points. K5 (spectators view the live scoreboard and market on their phones) and K6 (trading while the match is running), plus the additional context that traffic peaks during matches with hundreds of concurrent spectators. The scale is a planning assumption, not a measured user count.

Why it matters. Match traffic is not a gradual ramp. Registration opens and the market opens at kickoff; both create a step change. If the system cannot absorb the burst, spectators see timeouts or refused actions even though the business rules are correct.

SMART target.

Criterion	Target
Specific	Under a burst of 100 concurrent registration requests, the 95th-percentile response time is below 1,000 ms in the named CI environment, with no unhandled transport errors. After the burst, the system must not retain unbounded work or connections.
Measurable	The capacity test records p50, p95 and max latency and prints one JSON line. The post-peak resource condition is checked only where a deployment exposes resource metrics.
Assignable	Project owner (solo developer).
Realistic	100 concurrent clients are feasible in CI/local; 1,000 ms is a generous first target for a local PostgreSQL instance.
Time-related	First observed 2026-11-08; re-measured at each release and before the project submission.

Measurement method. `tests/invariants/tournament-capacity.test.ts` wraps each RPC in `performance.now()` and reports p50/p95/max. Capacity correctness (exactly 32 accepted) is a **Reliability** assertion; the elasticity assertion is only about time and resource behaviour.

Evidence. CI run 42435007354 observed p50 245.46 ms, p95 463.11 ms and max 479.64 ms. The final run 42435627680 observed p95 571.68 ms. Both are below the 1,000 ms target in that environment.

Status. Partially measured. Burst-response p95 is observed; the post-peak resource-release part of the course definition is **unverified** because there is no hosted deployment with autoscaling or resource metrics. The kata's "hundreds of concurrent spectators" remains an assumption, not a measurement.

Explicitly not claimed. The earlier "10,000 fans at kickoff" figure was an unsourced illustration and has been removed. No production elasticity claim is made.

3.3 Deployability

Course definition. Deployability is the ease, frequency and risk of deployment. The course's measure is how much human intervention is needed and how much time it takes (`SA_Lecture2`, `SA_Lecture5`).

Kata points. K1–K14 depend on increments reaching a running environment; the additional context states that a solo developer must deliver across two course deadlines. This is a project constraint rather than a single requirement.

Why it matters. A one-person project cannot afford a deployment process that consumes the time needed for domain work. A change that cannot reach a verified environment cannot produce evidence.

SMART target.

Criterion	Target
Specific	Merge to a running, verified local/CI environment in under 15 minutes. Schema changes are applied only through `supabase/migrations/`; no manual database step is allowed in a release.
Measurable	CI job duration, migration replay from an empty schema, application build/start, health check and smoke test. Manual steps are counted.
Assignable	Project owner (solo developer).
Realistic	The existing `Delivery and recovery` job already starts Supabase, replays migrations, builds, starts the app and runs smoke in roughly 2.5 minutes.
Time-related	Verified on every pull request and on `main`; current evidence from 2026-11-08.

Measurement method. The `Delivery and recovery` CI job performs the full local delivery path. `supabase db reset` replays every migration and the seed. `bun run smoke` checks health, login and the home shell. The operations runbook records the release, rollback and recovery steps.

Evidence. CI run 42467767299 completed the delivery and recovery job in about 2.5 minutes, including build, smoke and the public-schema backup/restore drill. The job applies migrations only from files and uses no manual database step. The operations runbook records the local commands.

Status. Measured for the local/CI baseline, not for production. There is no hosted deployment, TLS, DNS or production rollback drill. The "under 15 minutes" target is met by the CI job, but production deploy time is unmeasured.

3.4 Testability

Course definition. Testability is the ease and completeness of testing. The course's measures are code coverage and the time needed to run the tests (`SA_Lecture5`).

Kata points. K2, K4, K6–K8 and K10–K12 are concurrency rules. They cannot be verified by reading the interface; they need executable checks that can fail.

Why it matters. A green suite that cannot fail is not evidence. The project's architecture claims are only credible if the tests are failable and run on every change.

SMART target.

Criterion	Target
Specific	Every enforced domain invariant has an automated test that fails when the enforcing mechanism is deliberately weakened. The suite runs on every pull request.
Measurable	Test count, pass/fail count, fault-injection record and test execution time. The target is under 3 minutes for the suite.
Assignable	Project owner (solo developer).
Realistic	The current 28-test suite runs in about 8 seconds; the CI job around it completes in about 2.5 minutes.
Time-related	Verified on every pull request; current evidence from 2026-11-08.

Measurement method. `bun run test` runs the invariant suites against a real local Supabase/PostgreSQL database. Fault injection is recorded in the invariant registry: a constraint or guard is weakened, the suite must go red, then the migration is replayed to restore it.

Evidence. CI run 42435627680 passed 28 tests across 7 files, 0 failures, 256 expectations, in 8.28 seconds. The authorization suite covers anonymous, fan and organizer boundaries. Fault injection has been observed for INV-01, INV-02, M-INV4, M-INV6 and the M-INV7 gate.

Status. Measured for the recorded mechanisms. Five market invariants and the M-INV7 detail set still lack injection (FND-002); they are explicitly **Enforced, unverified**, not silently counted as proven.

3.5 Security — implicit cross-cutting characteristic

Course note. Security is not in the consolidated 12-item table. The course's implicit-characteristics slide uses security as the example of a requirement stakeholders rarely state but architects must make explicit (`SA_Lecture2`).

Kata points. K1 (staff manage matches and ticket sales), K13 (points redeem for rewards), K14 (club representatives access aggregated data). The points ledger and the club dataset create a trust boundary even though the kata does not use the word "security".

Why it matters. The system moves points and resolves markets. Without an authorization boundary, anyone with the public anon key could act as another participant or resolve a market. Club analytics also requires that aggregated data never exposes an individual.

SMART target.

Criterion	Target
Specific	No anonymous private read or mutation; no cross-user private read; only an organizer resolves a market; direct table writes are denied; the service-role key never reaches the browser or the application runtime.
Measurable	The authorization suite exercises the database as the anonymous role, two different fans and an organizer.
Assignable	Project owner (solo developer).

Criterion	Target
Realistic	The boundary is enforced in PostgreSQL with RLS, grants and `SECURITY DEFINER` RPCs; the suite runs in the same CI job.
Time-related	Verified in CI by 2026-11-07; re-checked at every authorization change and before the project submission.

Measurement method. `tests/invariants/authorization.test.ts` probes public and private reads, own-row isolation, organizer-only resolution, direct table writes, the identity-free public trade feed and owner-only trade requests. The mutation RPCs are `SECURITY DEFINER`, resolve `auth.uid()`, fix `search_path` and grant execute only to `authenticated`.

Evidence. CI run 42435627680 passed the authorization tests. The authorization verification record is `2026-11-95-authorization-rls.md`.

Status. Measured for the platform boundary. Production session hardening, rate limiting and the club-analytics privacy rule (aggregation thresholds, export controls) are **not implemented** and are stated as limits.

4. Traceability to kata points

Characteristic	Kata points	Why this point led to the characteristic	Quantified target	Evidence	Status
Reliability	K2, K4, K6–K8, K10–K12	Capacity, balances, positions and settlement are concurrency rules; a failure corrupts the ledger or overbooks a tournament.	100 concurrent registrations, capacity 32: 32 accepted, 68 refused, 0 transport errors; no negative balance; one-shot resolution; idempotent retries.	`tournament-capacity`, `market-concurrency`, `market-pricing`, `market-lifecycle`, `idempotency` suites; CI 42435627680.	Measured for recorded mechanisms; FND-002 open.
Elasticity	K5, K6, traffic context	Match-time traffic arrives as a step change at kickoff; spectators use phones during the match.	p95 < 1,000 ms for 100 concurrent registrations in CI; post-peak resource release unverified.	Capacity burst p50/p95/max; p95 463.11 ms and 571.68 ms.	Partially measured; post-peak resource behaviour unverified.
Deployability	K1–K14; solo-developer context	A one-person project across two deadlines must deliver increments	Merge to running verified local/CI environment <15 min; zero	`Delivery and recovery` job; runbook; recovery drill.	Measured for local/CI; production unverified.

Characteristic	Kata points	Why this point led to the characteristic	Quantified target	Evidence	Status
		without infrastructure work.	manual database steps; migrations only.		
Testability	K2, K4, K6–K8, K10–K12	The central rules cannot be verified by inspection; a green suite that cannot fail is not evidence.	Every enforced invariant has a failable test; suite <3 min; every PR.	28 tests, 0 failures, 8.28 s; fault injection for key mechanisms.	Measured; FND-002 open.
Security (implicit)	K1, K13, K14	Staff-only management, the points ledger and club privacy create a trust boundary.	No anonymous private read/mutation; no cross-user read; organizer-only resolution; direct writes denied; no service-role key in the client.	`authorization.test.ts`; CI 42435627680.	Measured; production hardening and club privacy unverified.

5. Deliberately not selected

Characteristic	Why not selected
Performance	Not selected as a separate characteristic. The only time target is the burst-response p95 under Elasticity. There is no separate throughput, CPU, memory or general page-latency target.
Responsiveness	Covered by the elasticity burst target. A general response-time target for every page would be invented, not derived from the kata.
Scalability	The kata states match-time peaks, not growth beyond one deployment. Elasticity covers the step change; scalability would need a growth requirement and a longer measurement window.
Fault tolerance	There is no hardware-failure or availability target. Reliability covers correctness under concurrent requests, which is the actual domain risk.
Modularity	Domain boundaries in PostgreSQL and a layered delivery shell are a design means to reliability and testability. No requirement forces independent module evolution or separate deployment.
Maintainability / Evolvability	The idempotency change demonstrates changeability, but the first increment has no separate maintenance or evolution target. Revisit if the club-analytics workload is extracted.
Simplicity	The modular monolith is chosen for simplicity, but the course offers no single direct metric beyond modularity metrics. It is a design rationale, not a selected characteristic.

Characteristic	Why not selected
Overall cost	The project is solo and time-boxed; cost is a constraint, not a quantified quality target.
Partitioning type	The system uses domain-oriented database modules with a layered delivery shell. This is a design decision, not a separately measured characteristic.

Selecting these later is cheap. Selecting them now without a driver would add complexity and cost for no stated benefit.

6. Trade-offs

Reliability vs Elasticity. The capacity guard takes a pessimistic lock on the tournament row, so concurrent registrations for the same tournament serialise. That reduces peak throughput in exchange for an invariant that is auditable in one place. The burst p95 remains below the target in the observed environment, but the trade is real: correctness is prioritised over maximum throughput.

Testability vs time to market. Writing tests that can fail costs time before a user-visible feature exists. The project accepts that cost for domain invariants only, because those are the rules where a silent failure is expensive. It does not extend the same rigour to every helper or presentation component.

Deployability vs control and cost. Supabase and the managed platform reduce infrastructure work, at the cost of vendor coupling and less control over connection limits and runtime behaviour. The local/CI baseline is verified; production remains a future decision.

Security vs simplicity. The mutation RPCs are `SECURITY DEFINER` because direct table writes are revoked. That creates a small, auditable mutation surface, but every function becomes a high-trust boundary: it must check `auth.uid()`, check the organizer role where required, validate input and fix `search_path`. RLS alone was not enough.

Modular monolith vs microservices. One deployable and one database are enough for the domain and the team of one. Microservices would add network failure modes, distributed consistency and operational work without a requirement that justifies them. The boundaries are enforced in the database/RPC surface and by the architecture guard, not by separate deployments.

No CQRS or event sourcing. A relational model with transactional functions satisfies every invariant currently stated. Event sourcing would add an operational surface, a projection rebuild path and a second failure mode without a requirement that needs it.

7. Evidence status and limits

Claim	Evidence	Limit
Capacity, balances, positions and resolution are correct under the recorded bursts	28 tests in CI; fault injection for key mechanisms	Local/CI, single database instance; five market invariants lack injection (FND-002)
Burst p95 is below 1,000 ms	463.11 ms and 571.68 ms observed	Environment-specific; post-peak resource release unverified
Delivery is repeatable from migrations	`Delivery and recovery` job; `supabase db reset`	Local/CI only; no production deployment
Backup and restore work for the application schema	67,009-byte public-schema restore; tables 10→10; RLS policies 10→10	Public schema only; no Supabase-managed auth/storage data; no production RTO/RPO
Authorization boundary is enforced	Authorization suite in CI	No production session/rate-limit hardening; club-analytics privacy rule not implemented
The tests can fail	Fault injection for INV-01, INV-02, M-INV4, M-INV6, M-INV7 gate	Remaining invariants have no observed-red injection

Explicitly not claimed. No production availability, no production latency, no real-user validation, and no independent human review. The kata's user numbers are planning assumptions. The system is not yet a complete product: ticket purchase, rewards redemption and club analytics are described in the kata but not all implemented.

8. Revision history

Date	Change
2026-10-19	First characteristic selection: Elasticity, Deployability, Testability, Integrity under concurrency, Security/authorization.
2026-11-07	Authorization/RLS implemented and verified. Integrity and security evidence strengthened.
2026-11-08	Idempotency, delivery health, smoke, p95 and recovery drill added.
2026-11-08	Course alignment revision: numbered kata requirements K1–K15; traceability table; SMART owner/time/review; Integrity under concurrency renamed Reliability; Security labelled implicit; Elasticity and Reliability targets separated; Performance/Responsiveness distinction documented; local deployability proxy added.

This revision supersedes the earlier naming and traceability, but it preserves the earlier evidence and limits. The history is a dated addendum, not a rewrite of what was observed.

9. Conclusion

MatchPulse selects **Reliability, Elasticity, Deployability, Testability and Security**. Reliability and Security are measured for the recorded mechanisms; Elasticity is partially measured because post-peak resource release is not observable without a deployment;

Deployability is measured for local/CI; Testability is measured with the stated injection gap. The remaining gaps are explicit: production deployment, post-peak resource behaviour, the remaining fault injections, real-user evidence and independent review. None is silently counted as achieved.